

CRATE ENGINE

engine no. CE-2.0 · factory assembled · macOS

A complete engineering team, shipped as one.

An orchestrated team — Coder, Reviewer, Designer, QA, and the Orchestrator you talk to — each sandboxed, each running on your own AI subscriptions. You install it with one line, direct it in plain English, and nothing merges without your explicit go.

SEAT 01

Orchestrator

your team lead —
the one you talk to

SEAT 02

Coder

implements, builds,
merges on your go

SEAT 03

Reviewer

adversarial code
review, read-only

SEAT 04

Designer

visual & UX design,
mobile-first

SEAT 05

QA

runtime testing,
screenshots,
honesty

One line installs everything — you bring the crew

Crate Engine ships the crate; the AI agents in it are **yours**. It never installs an agent and never signs into one — it detects what's already on your Mac and offers it for staffing. Your accounts stay yours, always.

BEFORE YOU INSTALL — ONCE PER AGENT, ITS OWN WAY
AGENT GET READY

claude run it in a terminal, finish its one-time setup, sign in with your Claude subscription, then /exit

pi run it, /login with ChatGPT (DeepSeek seats take its API key), quit

codex run it, sign in with your ChatGPT account, quit

Only the agents you want on the team — any subset works; the app offers what it finds.

THE INSTALL LINE · PASTE INTO TERMINAL, PRESS ENTER

```
curl -fsSL <your install link> | bash # the link ships with this manual
```

Four numbered steps run (cmux · node · the engine · the crate command — long ones print dots so you can see life), then a short “**Your AI agents**” report tells you exactly what was detected. Then the app launches itself: a cmux window appears and the app page fills it. **First-ever cmux launch may show a welcome window — click through it once.**

That's the only terminal command of day one. Everything from here is clicks.

The wizard: five screens, five clicks

WELCOME → STAFF → ATTACH → ARM → CHECK → BOOT

1

Staff your team

Click **Staff your team & start building**. The top card shows your detected agents; below, each seat has a tested default model with an honest note on how it bills (flat-rate on your subscription vs metered). Click **Save defaults** — set once, every project uses them — then

Continue → attach a project.

2

Attach a project

Attach a folder you already have, or **Start a new project** (Browse... has a + New folder button — no Finder needed). Click **Show me what will be added**, read the two honest lists (what goes in your repo, what stays local), then **Looks right — attach the team**.

3

Arm your team

One-time big downloads some seats need (e.g. QA's browser). If a box appears, click **Install now (recommended)** and watch the per-tool progress; otherwise it says “Nothing to download.” Then **Continue → final check**.

4

Final check

A wiring once-over. **Green** = good · grey = optional · **amber** shows the one line that fixes it. If an agent's sign-in has gone stale, this is where it's caught — before a dead boot, with the exact fix.

5

Boot the team

Click **Boot the team →**. A new cmux workspace opens with five panes and every seat boots its harness. The Orchestrator reads its brief, verifies its teammates, onboards them, and greets you: *“the rig is up — what shall we build first?”*

If Boot refuses: that's the app protecting you — the message names which agent isn't ready and exactly what to run. Fix it, click Boot again.

Talk to the Orchestrator — hold the gates

Click into the **Orchestrator pane** and say what you want, like a message to a team lead:

EXAMPLE DIRECTION

“Build me a small feature: add a footer line to the app's main page that says *Built by my AI team* with the current year. Run your full process and **hold at the merge gate** for my go.”

Then watch: the Coder implements on a branch, a build gate verifies it, and the Reviewer and QA check it **in parallel** — static review and runtime proof at once. You do nothing until the Orchestrator reports it's **HOLDING at the merge gate** with both verdicts.

YOUR ONE DECISION

Reviewer says **APPROVED** and QA says **PASS**? Type **“Merge it.”** — the only thing that ever merges code. An approval alone never does; the gate is yours.

THE RAILS YOUR TEAM RUNS ON

Verified before you no code reaches your gate without review + QA passing it

Sandboxed seats every seat works behind an OS-level wall — writes stay inside your project

Honest reporting seats refuse loudly and never fake evidence — a blocker is named, not papered over

An audit trail every state change lands in `.agents/state/events.log`

Stopping is nothing; coming back is one line

Everything lives on disk — the project, your staffing, the team's state. When you're done, just close the cmux window. Optionally tell the Orchestrator `.checkpoint` first to bank a tidy save.

EVENING

Stop

close the cmux window —
nothing else to save

`terminal ×0`



MORNING

Reopen

`cd <project> && crate
open`

lands on the Boot screen for
that project

`terminal ×1`



+1 CLICK

Boot

click **Boot the team** — five
panes return, oriented

`clicks ×1`

SEVERAL PROJECTS ON ONE MAC

`crate open` is **project-anchored**: run inside a project folder, it opens that project — never a guess. Run it from another project's folder and the app switches to it. A bare `crate open` anywhere else reopens your most recent project.

THE WELCOME SCREEN REMEMBERS TOO

With a project attached, Welcome shows “**Continue where you left off**” with the project path and a **Continue → Boot the team** button. You never re-walk the wizard for a project you already have.

The Boot screen is your live dashboard

After boot, the same screen watches every seat. Each row shows who's staffed, whether the pane is live, and how fresh its state file is.

THE THREE STATES YOU'LL SEE

DOT	MEANING	WHAT TO DO
● live	the seat's pane is up and running	nothing
● signed-out	the harness runs but its sign-in went stale (common after a machine sits overnight)	click Relaunch — a fresh launch picks up the renewed sign-in
● dead	the pane is gone (closed or crashed)	click Relaunch — a fresh pane comes back walled and oriented

BIGGER HAMMERS, STILL ONE CLICK

Refresh team (relaunch all)

relaunches every seat, Orchestrator last — the full morning-after reset

Update now (your customizations survive)

on Welcome when an engine update is available; fast-forwards the engine, checks your customizations, asks for one app restart

If the whole app window is gone (workspace closed, cmux quit): nothing is lost — `cd <project> && crate open` brings it back.

The whole command surface

Two commands are all a normal day needs. Everything else is a click — or a sentence to your Orchestrator.

REQUIRED · THE ONLY TWO

```
curl -fsSL <your install link> | bash # once, ever
```

```
cd <project> && crate open # every session
```

SAY IT TO THE ORCHESTRATOR (TYPED INTO ITS PANE)

plain English	the product itself — build, fix, redesign, “Merge it.”
.status	read-only rig health snapshot
.checkpoint	bank the work state before you stop
.resume / .recover	re-orient / rebuild the team after a mess

OPTIONAL TERMINAL TWINS (EVERY ONE HAS A BUTTON)

COMMAND	SAME AS
crate update	Welcome → Update now
crate doctor	the Final check screen
crate relaunch <seat> --workspace <ref>	dashboard → Relaunch

Plain answers to the five things that happen

YOU SEE	WHAT IT MEANS · WHAT TO DO
“REFUSING to boot: agent not ready”	An agent's sign-in isn't usable. The message names the agent and the exact fix — usually: run it once in a plain terminal, <code>/login</code> , and click Always Allow on any keychain dialog. Then Boot again.
A seat answers “API Error: 401 / run /login”	Its sign-in went stale mid-session (overnight is typical). <code>/login</code> in that pane, then use the dashboard's Relaunch on that seat — a running session can't pick up new credentials by itself.
The app window vanished	The workspace (or cmux) was closed. State is safe: <code>cd <project> && crate open</code> .
“has no .agents/bin — is the engine attached?”	The path isn't an attached project (typo, or a folder that never finished attach). Open the Attach screen and attach the real folder — attaching an already-attached project is safe and heals links.
“update available” on Welcome	Click Update now (your customizations survive) , then restart the app when asked (close the Crate Engine workspace, <code>crate open</code>).

A rule that runs through all of it: **Crate Engine refuses loudly and tells you the one line that fixes it.** If a screen ever lies, stalls, or leaves you stuck with no next step — that's a bug we want reported, not something to work around.